Coordination of Governed Agents Over Shared Context: Declarative Governance and Lyapunov-Based Finality

Submitted to arXiv

https://github.com/DealExMachina/swarm-of-governed-agents

February 28, 2026

Abstract

The emergence of LLM-powered autonomous agents has exposed the limits of pipeline-based coordination: rigid DAGs cannot accommodate contradiction, evolving evidence, or formal convergence. We propose *declarative governance over shared immutable context*, a paradigm where autonomous agents reason over a common semantic graph, propose state transitions, and reach consensus through formal convergence mechanisms—without predefined execution sequences. The architecture rests on four principles: (1) an append-only event log and a bitemporal CRDT-inspired semantic graph providing shared, monotonic, temporally auditable context; (2) a pluggable declarative policy engine separating governance from agent implementation; (3) Lyapunov-based finality tracking with monotonicity gates, plateau detection, and human-in-the-loop routing; (4) fine-grained relationship-based access control governing agent execution, document access, and policy modification. We ground this design in a theoretical framework combining Whitney-stratified constraint manifolds—where governance rules progressively reduce a high-dimensional enterprise decision space to discrete terminal states—with Fisher–Rao information geometry for convergence dynamics. Unlike orchestration frameworks (AutoGen, CrewAI, LangGraph) that wire agents into predefined topologies, our system lets agents independently reason over shared state and propose actions evaluated by declarative policy. Validation on a realistic M&A due-diligence scenario demonstrates convergence in 12 rounds across 5 contradictory documents, with 83% autonomous contradiction resolution and complete audit trail. The resulting architecture is designed for regulated environments where temporal auditability, policy traceability, and formal convergence guarantees are operational requirements.

1 Introduction

The rapid maturation of LLM-powered agents—capable of reasoning, planning, and tool use—has created a coordination problem that traditional workflow engines were not designed to solve. Modern agents do not merely execute predefined tasks; they interpret context, generate hypotheses, and produce claims that may contradict each other. This capability demands a fundamentally different coordination model.

Current approaches fall into two categories, both inadequate:

Pipeline orchestration (Airflow, Temporal, Prefect) assumes static task graphs executed in topological order. When agents produce contradictory outputs, the pipeline either silently overwrites earlier results or fails entirely. There is no mechanism for tracking disagreement, no formal convergence guarantee, and no governance trail explaining *why* a particular conclusion was reached.

Agent frameworks (AutoGen, CrewAI, LangGraph) represent progress: they enable multi-agent conversations, role-based delegation, and tool use. However, they still wire agents into predefined topologies—sequential chains, hierarchical delegation, or cyclic graphs with hardcoded routing. Coordination logic is embedded in code, not governed by policy. There is no shared immutable context, no formal finality mechanism, and no fine-grained access control.

We propose a third path: *declarative governance over shared context.* Rather than sequencing agents, the system maintains:

1. **Shared immutable context:** An append-only event log and a CRDT-inspired semantic graph where confidence scores ratchet upward, contradictions are tracked explicitly, and no state is ever deleted—only superseded.

2. **Declarative governance:** Agent activation and state transitions are governed by human-readable rules. No agent directly modifies shared state; all propose transitions that governance evaluates before an executor implements them.

3. **Formal convergence:** A Lyapunov disagreement function $V(t)$ tracks distance to targets across weighted dimensions. Monotonicity gates, plateau detection, and divergence escalation provide formal guarantees. When autonomous convergence stalls, the system routes to structured human review with full context.

4. **Fine-grained access governance:** Relationship-based access control governs who can read documents, propose transitions, approve decisions, and modify policies.

This paper presents the formal foundations, system design, and experimental validation of this paradigm. Section 2 reviews related work. Section 3 formalizes the problem with explicit definitions, a monotonicity theorem, and a threat model. Sections 4–5 develop the theoretical framework. Section 6 describes the system architecture. Section 7 presents prototype validation results. Section 10 defines five experimental protocols for further evaluation. Section 9 states explicit scope boundaries. Implementation details are provided in the Appendix.

2 Related Work

2.1 Agent Coordination Frameworks

AutoGen [23] introduces conversable agents with role-based chat patterns. Coordination is conversation-driven: agents exchange messages in predefined patterns (sequential, group chat, nested). However,

coordination topology is hardcoded, there is no shared persistent state between conversations, and no formal convergence mechanism.

CrewAI [14] organizes agents into crews with roles, goals, and tools. A hierarchical manager delegates tasks. Coordination follows a fixed process (sequential or hierarchical), with no mechanism for handling contradictions between agent outputs or tracking disagreement over time.

LangGraph [10] models agent coordination as state machines with explicit edges and conditional routing. This is closer to our approach—state is explicit and transitions are defined—but routing logic is embedded in code, not declarative policy. There is no immutable audit log, no CRDT semantics, and no formal convergence tracking.

All three frameworks assume that coordination topology can be predefined. Our system replaces topology with policy: agents reason independently over shared state, propose transitions, and governance rules determine what happens next.

2.2 Consensus and Convergence Theory

Olfati-Saber and Murray [15] established the Lyapunov consensus framework for multi-agent systems, proving that monotonically decreasing disagreement functions guarantee convergence under mild assumptions. We adapt this to knowledge-work coordination, defining a weighted disagreement function over semantic dimensions.

Ruan et al. [18] formalized the multi-agent refinement problem for reasoning LLM agents in the Aegean protocol, providing provable safety and liveness guarantees with quorum-based consensus and early termination. Their monotonicity gates—requiring β consecutive stable rounds before declaring convergence—directly inform our finality mechanism. EMA-based plateau detection, also employed in Aegean, identifies systems that stall despite apparent activity; we adopt this for our stagnation routing.

De la Chica Rodriguez and Vera Díaz [3] proposed Self-Evolving Coordination Protocols (SECP) for multi-agent systems in regulated financial environments, evaluating six Byzantine consensus protocols under hard constraints including Byzantine fault tolerance ($f < n/3$), bounded message complexity, and complete non-statistical safety arguments. Their work demonstrates that formal governance invariants can coexist with protocol adaptability—a concern directly relevant to our declarative governance layer.

Li et al. [7] proposed market-making as a coordination framework for multi-agent LLM systems, where agents trade probabilistic beliefs to converge toward shared outcomes. This approach parallels our information-geometric interpretation (Section 4.2): agents update belief distributions, and convergence corresponds to agreement on a target distribution.

2.3 CRDT Semantics and Monotonic State

Laddad et al. [8] synthesized verified CRDTs with monotonic merge operations (Katara), proving that semilattice-based state compositions guarantee commutativity and monotonicity by construc-

tion. Our semantic graph adopts this principle: confidence scores ratchet upward only, contradictions once marked are irreversible, and nodes are staled rather than deleted.

Shen et al. [19] applied CRDT-based coordination to multi-agent LLM code generation (CodeCRDT), demonstrating that observation-driven coordination—where agents monitor shared state rather than exchange messages—achieves strong eventual consistency with zero merge conflicts. This validates the scalability of CRDT semantics for LLM agent coordination.

2.4 Governance and Policy Enforcement

Recent work on Governance-as-a-Service [6] proposes a modular, policy-driven enforcement layer that regulates agent outputs at runtime through declarative rules and trust scoring, without altering model internals. While GaaS focuses on output filtering, our approach integrates governance into the coordination substrate itself: governance rules determine state transitions, not just output validity.

Pang et al. [16] formalized Zanzibar-style relationship-based access control, implemented in OpenFGA. We integrate this for fine-grained governance of document access, transition approval, and policy modification.

2.5 Stigmergic Coordination

Dorigo et al. [4] formalized stigmergic coordination in swarm intelligence: agents respond to environmental signals rather than direct communication. Our pressure-directed activation mechanism applies this principle—agents are routed toward high-pressure dimensions without centralized scheduling.

3 Problem Setting and Formal Definitions

We first formalize the coordination problem and establish definitions used throughout the paper. The threat model and complexity bounds clarify the scope of our claims.

3.1 Shared Context Graph

Definition 1 (Shared Context Graph). A shared context graph is a directed labeled graph $G = (N, E, \sigma, \tau)$ where N is a set of typed nodes $n_i \in \{\textsc{Claim}, \textsc{Goal}, \textsc{Risk}, \textsc{Contradiction}\}$, $E \subseteq N \times N$ is a set of typed edges, $\sigma : N \to [0, 1]$ assigns a confidence score to each node, and $\tau : N \to \mathbb{R}^2 \times \mathbb{R}^2$ assigns bitemporal coordinates (valid time interval, transaction time interval) to each node. The graph satisfies three monotonicity invariants:

1. *Confidence monotonicity:* $\sigma(n_i, t+1) \geq \sigma(n_i, t)$ for all n_i, t.
2. *Contradiction irreversibility:* if $(n_i, n_j) \in E$ with type $\textsc{Contradicts}$ at time t, then $(n_i, n_j) \in E$ for all $t' > t$.
3. *No deletion:* $|N(t+1)| \geq |N(t)|$ and $|E(t+1)| \geq |E(t)|$. Superseded nodes are staled, not removed.

3.2 Governance Function

Definition 2 (Governance Function). A governance function is a map $\Pi : \mathcal{P} \times G \times \mathcal{R} \to \{\textsc{Approve}, \textsc{Reject}, \textsc{Escalate}\}$, where $\mathcal{P}$ is the set of agent proposals (candidate state transitions with justification), G is the current shared context graph, and $\mathcal{R}$ is a set of declarative rules. Π is *deterministic*: given the same $(p, G, \mathcal{R})$, it always produces the same decision. Π is *auditable*: every evaluation is recorded as an immutable event.

3.3 Convergence and Finality

Definition 3 (Lyapunov Disagreement Function). Given k weighted convergence dimensions $\mathcal{D} = \{(d_i, w_i, \text{target}_i)\}_{i=1}^{k}$ with $\sum w_i = 1$, the disagreement function is:

$$V(t) = \sum_{i=1}^{k} w_i \cdot (\text{target}_i - \text{actual}_i(t))^2 \geq 0$$

The system is *converging* at time t if $V(t) < V(t-1)$, *stalled* if $|V(t) - V(t-1)| < \delta$ for τ consecutive rounds, and *diverging* if $V(t) > V(t-1) + \epsilon$.

Definition 4 (Finality Predicate). The system reaches finality at time t when the conjunction of five gates is satisfied:

$$F(t) = \bigwedge_{g \in \{A,B,C,D,E\}} \text{Gate}_g(t)$$

where:

- Gate A (Monotonicity): $V(t-j) \geq V(t-j+1)$ for $j = 1, \ldots, \beta$.
- Gate B (Evidence coverage): all required evidence types present and within staleness bounds.
- Gate C (Oscillation): lag-1 autocorrelation of $V(t)$ below threshold.
- Gate D (Quiescence): no new proposals for τ_q consecutive rounds.
- Gate E (Minimum content): $|N| \geq n_{\min}$ and all declared goals have associated claims.

3.4 Monotonicity Theorem (Sketch)

Theorem 1 (Conditional Monotonicity of $V(t)$). Under the three monotonicity invariants of Definition 1 (confidence monotonicity, contradiction irreversibility, no deletion), if every agent proposal that is approved by Π either (a) increases some $\sigma(n_i)$, (b) resolves an existing contradiction, or (c) advances a goal, then $V(t+1) \leq V(t)$.

Proof sketch. Each dimension of $V(t)$ measures a squared gap to target. Condition (a) reduces the claim confidence gap. Condition (b) reduces the contradiction count gap. Condition (c) reduces the goal completion gap. The monotonicity invariants ensure no approved action can increase any gap: confidence cannot decrease, contradictions cannot reappear, and nodes cannot be deleted. The risk dimension is bounded by the other three: resolved contradictions and increased confidence mechanically reduce residual risk. Therefore $V(t+1) \leq V(t)$. □

Corollary 1 (Termination). Since $V(t) \geq 0$ and is non-increasing under approved transitions, and the system declares BLOCKED after τ_b idle cycles or EXPIRED after a time limit, the system always terminates in finite time—either at a finality state (RESOLVED, HITL Review) or at a boundary state (BLOCKED, EXPIRED, ESCALATED).

3.5 Threat Model and Complexity

Threat model. All agents are assumed cooperative (non-Byzantine). Agents may be slow, fail, or produce incorrect outputs, but they do not deliberately subvert the protocol. Byzantine fault tolerance is an explicit non-goal of this work; we refer to [3] for BFT extensions in regulated environments.

Message complexity. The three-node state cycle with epoch-based CAS requires $O(n)$ governance evaluations per round (one per agent proposal), yielding $O(n \cdot k)$ total evaluations for k convergence rounds. This contrasts with the $O(n^2)$ message complexity of BFT protocols [3, 2], though the two are not directly comparable: our system trades Byzantine tolerance for lower coordination overhead.

4 Theoretical Foundations

A regulated enterprise maintains n decision parameters—financial metrics, compliance statuses, risk assessments, contractual terms, personnel evaluations. Each parameter spans a continuous range of possible values, and the full decision space is a high-dimensional configuration space $\mathcal{C} \subseteq \mathbb{R}^n$. Governance rules, however, admit only a strict subset of configurations. The central theoretical question is: *how do governance constraints reduce a continuous, high-dimensional space to a discrete set of permitted terminal states?*

We formalize this using stratified constraint manifolds (Section 4.1) for the structure of permitted states, and information geometry (Section 4.2) for the dynamics of convergence toward those states. This framework is dimension-agnostic: it scales to arbitrary n without structural modification.

4.1 Constraint Manifolds and Whitney Stratification

Let $\mathcal{C} = \mathbb{R}^n$ denote the full configuration space of an enterprise decision process, where each coordinate x_i represents a decision parameter. Without governance, agents may propose any point in $\mathcal{C}$.

Each governance rule r_k imposes a constraint—an equation $g_k(x) = 0$ or an inequality $h_k(x) \leq 0$—defining a *permitted region*. The intersection of all m active governance constraints defines a *semi-algebraic set* [22]:

$$\mathcal{S} = \{x \in \mathcal{C} \mid g_k(x) = 0,\ h_j(x) \leq 0,\ k \in K,\ j \in J\}$$

By the Whitney stratification theorem [22, 17], $\mathcal{S}$ decomposes into a finite disjoint union of smooth manifolds (strata) of decreasing dimension:

$$\mathcal{S} = \bigsqcup_{i=0}^{p} S_i, \quad \dim(S_0) > \dim(S_1) > \cdots > \dim(S_p) = 0$$

where each stratum S_i is a smooth manifold, and the closure of each stratum contains only strata of equal or lower dimension (the *frontier condition*).

Interpretation for governed agents. The stratification captures the progressive narrowing of permitted states as governance constraints accumulate:

- **Top stratum** S_0: the broadest permitted region, where few constraints are active. Agents operate with maximal freedom.
- **Intermediate strata** $S_1, \ldots, S_{p-1}$: as evidence accumulates and additional governance rules activate (contradiction detected, risk threshold crossed, compliance rule triggered), the system transitions to strata of lower dimension. Each activated constraint reduces the degrees of freedom by at least one.
- **Terminal strata** S_p ($\dim = 0$): isolated points representing fully determined, governance-compliant terminal states. These are the *finality states*—discrete configurations where all constraints are simultaneously satisfied and no degrees of freedom remain.

Governance modes as constraint regimes. Different governance modes correspond to different constraint densities on $\mathcal{S}$. A permissive mode activates minimal constraints, keeping $\dim(\mathcal{S})$ high; a strict mode activates maximal constraints from the start, forcing agents into a tightly constrained space; an intermediate mode progressively adds constraints through human review at each approval step.

Scalability in n dimensions. Whitney stratification and semi-algebraic geometry are dimension-agnostic: the Tarski–Seidenberg theorem guarantees that projections and Boolean combinations of semi-algebraic sets remain semi-algebraic, regardless of n [21]. Adding new decision parameters or new governance rules does not change the mathematical framework—it only modifies the specific stratification of $\mathcal{S}$. This ensures theoretical scalability from prototype to enterprise-scale deployments with thousands of parameters.

4.2 Information-Geometric Dynamics on Constrained Manifolds

The stratified manifold $\mathcal{S}$ describes the *structure* of permitted states. We now address the *dynamics*: how do agents move through $\mathcal{S}$ toward terminal strata?

Each agent a maintains an internal belief state—a probability distribution $p_a(\theta)$ over a parameter space Θ representing its assessment of the decision variables. The collection of all such distributions forms a *statistical manifold* $\mathcal{M}$ equipped with the Fisher–Rao metric [1]:

$$g_{ij}(\theta) = \mathbb{E}\left[\frac{\partial \log p(x|\theta)}{\partial \theta_i} \cdot \frac{\partial \log p(x|\theta)}{\partial \theta_j}\right]$$

This Riemannian metric induces a natural geometry on the space of agent beliefs: distances between distributions correspond to distinguishability, and geodesics represent the most efficient belief updates.

Governance as sub-manifold constraint. Governance rules restrict the agent belief space to a sub-manifold $\mathcal{M}_{\mathcal{S}} \subset \mathcal{M}$: only distributions consistent with the current stratum of $\mathcal{S}$ are permitted. As constraints activate and the system moves to lower strata, $\mathcal{M}_{\mathcal{S}}$ narrows correspondingly.

Convergence as constrained geodesic flow. The Lyapunov disagreement function $V(t)$ (Section 5.2) can be interpreted information-geometrically as a weighted divergence between the current collective belief state and the target distribution at a terminal stratum. Convergence corresponds to geodesic flow on $\mathcal{M}_{\mathcal{S}}$ toward the zero-dimensional strata of $\mathcal{S}$:

$$V(t) \sim D_{\mathrm{KL}}(p_{\mathrm{collective}}(t) \,\|\, p_{\mathrm{target}}) \to 0$$

Plateau detection identifies when the flow stalls on a stratum boundary—a singularity of the stratified space where the agent system requires additional information (or human input) to transition to a lower stratum.

Pressure-directed activation as gradient flow. The stigmergic routing mechanism—directing agents toward the highest-residual-gap dimension—corresponds to following the steepest descent direction of $V(t)$ on $\mathcal{M}_{\mathcal{S}}$. In information-geometric terms, agents are routed along the natural gradient [1], which accounts for the curvature of the statistical manifold and is invariant to reparametrization.

4.3 Philosophical Antecedent: Margulis Rigidity

The theoretical framework above—constraints progressively reducing a continuous space to discrete terminal states—echoes a deep theme in mathematics: Margulis's rigidity theorems [11, 12, 13, 5]. Margulis proved that for lattices in semisimple Lie groups of rank ≥ 2, local flexibility vanishes: representations are rigid, lattices are arithmetic (hence discrete and enumerable), and no intermediate normal subgroups exist.

The conceptual parallel is suggestive: in our system, multiple independent governance constraints force the permitted configuration space to be discrete—just as multiple independent symmetry constraints force lattices to be rigid. The normal subgroup theorem's dichotomy (either near-universal compliance or system-wide failure, with no stable intermediate) mirrors the empirical observation that partially governed agent systems are unstable.

We invoke Margulis as philosophical motivation, not as formal machinery. The operative mathematical tools are Whitney stratification (for the static structure) and Fisher–Rao geometry (for the dynamics). Margulis's contribution is the deep intuition that sufficiently constrained algebraic structures cannot deform continuously—they are forced into discrete configurations. This intuition guides the design principle that enterprise governance must be multi-dimensional (multiple independent constraint axes) to achieve the rigidity required for audit, compliance, and reproducibility.

5 Convergence Theory and Finality

5.1 The Finality Problem

Traditional systems declare finality by threshold: "if confidence > 0.95, done." This is brittle: contradictions may emerge post-threshold, systems cycle indefinitely, and convergence is unguaranteed. Our stratified constraint framework (Section 4.1) provides a structural resolution: finality corresponds to reaching a zero-dimensional stratum of the permitted space $\mathcal{S}$—a state where all governance constraints are satisfied and no degrees of freedom remain.

5.2 Lyapunov Disagreement Function

Following Olfati-Saber and Murray [15], we define a scalar disagreement function $V(t) \geq 0$ measuring distance to convergence targets:

$$V(t) = \sum_{d \in \mathcal{D}} w_d \cdot (\text{target}_d - \text{actual}_d(t))^2$$

where $\mathcal{D}$ comprises k weighted dimensions representing orthogonal aspects of the decision process (e.g., claim confidence, contradiction resolution, goal completion, residual risk). The weights w_d reflect relative importance and sum to unity.

The convergence rate α is computed as:

$$\alpha = -\ln\left(\frac{V(t)}{V(t-1)}\right)$$

with estimated time to arrival:

$$\text{ETA} = \left\lceil \frac{-\ln(\epsilon/V(t))}{\alpha} \right\rceil$$

Three regimes emerge: $\alpha > 0$ (converging, finite ETA), $\alpha \approx 0$ (stalled, plateau detection activates), and $\alpha < 0$ (diverging, triggers escalation).

5.3 Finality Mechanisms

Monotonicity Gate [18]. The goal score must remain non-decreasing for β consecutive rounds before auto-finality eligibility. Any regression resets the counter. This prevents premature finality declarations based on transient spikes.

Plateau Detection [18]. An exponential moving average of the progress ratio identifies systems that stall despite apparent activity. Plateau is declared when progress falls below a threshold for τ consecutive rounds. In our stratification framework, plateaus correspond to the system reaching a stratum boundary where it cannot progress without additional constraints or information.

Pressure-Directed Activation [4]. Agents are routed toward the dimension with highest residual gap (bottleneck), without centralized scheduling. This stigmergic mechanism ensures resources concentrate where they matter most, corresponding to natural gradient descent on the constrained belief manifold (Section 4.2).

Divergence Escalation. If α becomes sufficiently negative (disagreement increasing), the system transitions to an escalated state requiring human intervention.

Human-in-the-Loop Review. When the goal score enters a near-finality range and plateau is detected, the system queues a structured review with: dimension breakdown, blocking factors, generated explanation, and suggested actions (approve finality, provide resolution, escalate, or defer).

5.4 Finality States

The system defines six terminal conditions:

State	Trigger	Action
RESOLVED	High score + monotonic gate passed	Automatic completion
ACTIVE	Score or gate not satisfied	Continue processing
ESCALATED	High risk, many contradictions, or divergence	Human intervention
BLOCKED	Prolonged idle cycles without updates	Stalled
EXPIRED	No activity beyond time limit	Process expired
HITL Review	Near-finality score + plateau	Structured human decision

Table 1: Finality states and their triggers.

6 System Design

The theoretical framework of Sections 4–5 motivates four design principles, which together define the system architecture. Implementation details (schemas, configurations, exact parameters) are provided in Appendix A.

6.1 Principle 1: Proposals, Approval, Execution

The central invariant is: *no agent directly modifies shared state.* Each agent reads shared context, reasons about it, and emits a proposal (a candidate state transition with justification). Declarative rules determine whether the proposal is approved, blocked, or routed to human review. Only approved proposals are applied to shared state via atomic transitions.

This pattern provides complete auditability: every state change has a proposal, an evaluation, and an execution record. It also enables governance without code changes: rules are configuration, not compiled logic.

6.2 Principle 2: Shared Immutable Context

All agents read from and write to two shared data structures:

Append-Only Event Log. Every claim, fact, and agent decision is recorded as an immutable event. Events are never modified or deleted. This provides tamper-proof auditability, full reproducibility (replay the log to reconstruct any state), and causal traceability (which facts triggered which agent actions).

CRDT-Inspired Semantic Graph. A semantic graph maintains relationships with monotonic guarantees [8]: confidence scores ratchet upward only, contradictions once marked are irreversible, and superseded nodes are staled rather than deleted. These CRDT semantics prevent state regression: the graph can only move toward resolution, never away from it.

The semantic graph implements *bitemporal* tracking [20]: every node and edge carries both *valid time* (when a fact holds in the real world) and *transaction time* (when the system recorded it). This dual temporality enables point-in-time reconstruction for audit ("What did the system know at date T' about facts valid at date T?") and prevents false contradictions from temporal misalignment.

6.3 Principle 3: Declarative Governance

Agent activation and state transitions are defined by human-readable declarative rules, not compiled code. Three governance modes provide different compliance profiles:

- **Permissive mode:** Automatic approval for valid transitions. Suitable for low-risk workflows. In the stratification framework, this corresponds to minimal active constraints ($\dim(\mathcal{S})$ remains high).
- **Supervised mode:** All proposals route to human review. Each approval step adds constraints, progressively reducing the permitted space.
- **Deterministic mode:** Rule-based decisions without LLM rationale. Maximal constraints active from the start, forcing agents into a tightly constrained space.

Per-scope overrides allow mixing modes within a single system. The system operates on a tight three-node state cycle (ingest $\rightarrow$ extract $\rightarrow$ check $\rightarrow$ ingest), where each transition requires governance approval and only one agent succeeds per cycle via atomic epoch-based compare-and-swap. This prevents race conditions and ensures every cycle produces a complete audit record.

6.4 Principle 4: Fine-Grained Access Governance

Relationship-based access control [16] enforces authorization at four levels: document access, transition approval, policy modification, and audit visibility. This ensures that governance is not only about *what* agents can do, but *who* can authorize it.

6.5 Agent Architecture

Six agents operate independently within governance constraints:

Facts Agent. Extracts claims, goals, and risks from unstructured context. Outputs typed nodes to the semantic graph with confidence scores and source references.

Drift Agent. Computes changes from baseline across drift types (contradiction, goal misalignment, factual inaccuracy, entropy). Classifies severity and routes to governance for policy evaluation.

Planner Agent. Synthesizes facts and drift into ranked proposals. Maps drift dimensions to required next actions and proposes state transitions with justification.

Status Agent. Tracks $V(t)$, α, monotonicity gate, and plateau detection. Routes near-finality cases to human review with full convergence analysis.

Governance Agent. Evaluates all proposals against declarative rules. Routes through the appropriate approval mode. Enforces access control.

Executor. Implements approved actions via atomic state transitions. Records in the append-only event log. Updates the semantic graph with monotonic upserts.

Key invariant: Agents do not call each other directly. They emit events that trigger governance rules. This decoupling enables resilience (agent failure does not cascade), auditability (every interaction is logged), and scalability (agents can be independently scaled).

6.6 Comparison with Agent Frameworks

Property	AutoGen	CrewAI	LangGraph	Ours
Coordination model	Chat patterns	Hierarchical	State machine	Declarative policy
Shared persistent state	No	No	Partial	Yes (WAL + graph)
Formal convergence	No	No	No	Yes (Lyapunov)
Contradiction tracking	No	No	No	Yes (semantic graph)
Immutable audit log	No	No	No	Yes (append-only)
Declarative governance	No	No	No	Yes
Fine-grained access control	No	No	No	Yes
CRDT monotonic semantics	No	No	No	Yes
Human-in-the-loop finality	Manual	Manual	Manual	Structured + context
Agent topology	Predefined	Predefined	Predefined	Emergent (policy)

Table 2: Comparison with existing agent coordination frameworks.

The fundamental difference is architectural: existing frameworks *wire agents into topologies* (chains, trees, graphs) where coordination is embedded in code. Our system *governs agents through policy over shared state*, allowing coordination patterns to emerge from rules rather than being predetermined. When business requirements change, our system requires rule edits; existing frameworks require code changes, testing, and redeployment.

7 Validation: Project Horizon

7.1 Scenario: M&A Due Diligence

Project Horizon simulates due diligence on NovaTech AG, a B2B SaaS firm in supply chain compliance. Five documents are ingested sequentially, each introducing new facts and contradictions:

1. **Analyst Briefing:** Baseline facts (EUR 50M ARR, 7 patents, 45% CAGR). Low finality score.

2. **Financial DD:** ARR revised to EUR 38M—a EUR 12M discrepancy. High contradiction drift triggers governance block, preventing the cycle from advancing silently.

3. **Technical Assessment:** CTO and 2 senior engineers departing. Medium drift escalates to risk committee review.

4. **Market Intelligence:** Patent infringement suit filed. Multiple unresolved contradictions push toward ESCALATED finality state.

5. **Legal Review:** Partial contradiction resolution. Goal score crosses near-finality but remains below auto-finality threshold. HITL review queued with structured decision options.

7.2 Key Governance Moments

- **Cycle blocked after Document 2:** Financial discrepancy is not silently absorbed. Governance rules prevent advancement until contradiction is addressed.
- **Investigation recommended:** Contradiction drift signals formal investigation.
- **Risk escalation:** Critical personnel departures routed to structured escalation path.
- **Near-finality HITL:** System has sufficient data to recommend but insufficient confidence for autonomous decision. Human receives dimension breakdown, blockers, and suggested actions.

This progression illustrates the stratification framework in action: from a high-dimensional space of possible assessments (Document 1, few constraints), through progressive narrowing (Documents 2–4, contradictions and risks activating governance rules), to a near-discrete set of permitted conclusions (Document 5, HITL review of the remaining degrees of freedom).

7.3 Results

Metric	Value
Input documents	5 (analyst, financial, technical, market, legal)
Facts extracted	312
Contradictions identified	18
Agent-resolved contradictions	15 (83%)
Contradictions routed to human	3 (17%)
Total convergence rounds	12
Wall-clock time	240 sec
Immutable audit events	487
Policy rule evaluations	156
Governance blocks triggered	3

Table 3: Project Horizon results.

7.4 Known Validation Gaps

The project explicitly acknowledges: no stress testing for high throughput, no chaos engineering for service failures, single-scope E2E only, synthetic convergence data (not real LLM trajectories), and unmeasured LLM oversight quality. Testing and benchmark details are provided in Appendix A.6.

8 Discussion

8.1 Enterprise Applicability

Compliance and auditability. Regulatory frameworks (SOX, GDPR, HIPAA) require demonstrating who made decisions, when, and based on what evidence. In our system, governance rules

are human-readable, versioned, and approved. All claims and decisions reside in the immutable event log. The bitemporal semantic graph enables auditors to query: "Who decided this value, and what documents justified that conclusion at that point in time?"

Operational resilience. Agent failures do not cascade. When one agent is slow or fails, others continue operating. The disagreement metric $V(t)$ rises, triggering escalation rules or human routing. Atomic transitions ensure consistency even under concurrent agent execution.

Governance agility. When business needs change, governance rules are updated declaratively. The change is auditable (versioned), testable (new rules are evaluated against the existing event log), and live immediately. No code recompilation, no redeployment.

8.2 The Reasoning Agent Paradigm

The emergence of reasoning-capable LLM agents—systems that can plan, hypothesize, and revise conclusions based on new evidence—fundamentally changes the coordination problem. In the DAG paradigm, agents are execution units: they receive inputs and produce outputs in a predefined sequence. In the reasoning paradigm, agents are *epistemic actors*: they maintain beliefs, generate hypotheses, and update them as evidence accumulates.

This shift has three implications for coordination:

Contradiction is information, not failure. When two reasoning agents disagree about a fact, this disagreement is valuable signal. A governance-based system tracks contradictions explicitly, routes them to investigation, and declares finality only when they are resolved—or escalated to human judgment with full context.

Coordination must be emergent, not hardcoded. Reasoning agents discover new information that may invalidate the planned execution path. A governance-based system allows agents to propose new directions, evaluated by declarative rules rather than predetermined sequences.

Finality requires formal guarantees. When agents reason and revise, the system may never converge without formal mechanisms. Lyapunov-based convergence tracking provides mathematical guarantees that the system either reaches consensus or explicitly routes to human review—it cannot cycle indefinitely.

As LLM agents become more capable, the governance-based coordination model becomes more relevant, not less.

9 Scope and Boundaries

Following the methodological transparency advocated by de la Chica Rodriguez and Vera Díaz [3], we explicitly state the boundaries of our claims.

9.1 What This Paper Demonstrates

1. **Architectural feasibility** of declarative governance over shared immutable context, with complete separation of policy from agent implementation.

2. **Formal convergence tracking** via a Lyapunov disagreement function with five gates that prevent premature finality (Theorem 1).

3. **Theoretical grounding** via Whitney-stratified constraint manifolds and Fisher–Rao information geometry, providing a dimension-agnostic framework for governance-induced discretization.

4. **Prototype validation** on a realistic M&A scenario with 5 documents, 312 facts, 18 contradictions, converging in 12 rounds with 83% autonomous resolution.

5. **Complementarity with SECP**: our convergence and finality mechanisms address four of the six open questions identified in [3]—multi-iteration dynamics, invariant preservation, audit infrastructure, and complexity bounding.

9.2 What This Paper Does Not Demonstrate

1. **Statistical validation under real LLM stochasticity.** Convergence benchmarks use synthetic trajectories. Real LLM reasoning traces exhibit stochasticity that may violate Theorem 1's preconditions.

2. **Byzantine fault tolerance.** All agents are assumed cooperative (Section 3). Byzantine resilience requires the BFT machinery explored in [3, 2].

3. **Scalability beyond prototype.** Validation covers 50 claims, 5 documents, 6 agents. Scaling behavior at 10^3–10^4 claims is unknown.

4. **Machine-checked proofs.** Theorem 1 is a proof sketch, not a mechanized verification. Formal proof in Coq or Lean remains future work.

5. **Real human-in-the-loop validation.** The HITL routing mechanism is designed and implemented but not empirically evaluated with actual human reviewers.

6. **Coverage-autonomy trade-off formalization.** SECP's central finding—that coverage and autonomy are in tension [3]—is not formalized in our framework. Our governance modes (YOLO/MITL/MASTER) navigate this trade-off empirically, not theoretically.

7. **Protocol self-modification.** Unlike SECP's self-evolving protocols, our governance rules are static YAML. Adaptive governance is future work.

10 Proposed Experimental Protocol

Beyond the Project Horizon prototype validation, we define five experimental protocols designed to address open questions in the literature—particularly those identified by [3]. These experiments are designed to be reproducible using the reference implementation.

Experiment 1: Multi-Iteration Convergence Dynamics. Run 20 convergence cycles on a fixed scope with incremental context injection, varying contradiction density (0, 1, 3, 5 contradictions per injection). Measure $V(t)$ trajectory, $\alpha(t)$ convergence rate, and gate satisfaction per round. Expected outcome: characteristic trajectory shapes (exponential decay for easy cases, plateau-then-resolution for hard cases, oscillation-then-escalation for irreconcilable conflicts). This directly addresses SECP's open question on multi-iteration convergence dynamics, which their single-iteration design could not evaluate.

Experiment 2: Scalability. Vary claims (10, 50, 100, 500, 1000), contradiction rate (10%, 30%, 50%), and agent count (3, 5, 7, 12). Measure rounds to convergence, wall-clock time, LLM token consumption, and audit event count under fixed governance mode. Expected outcome: identification of the scaling bottleneck (likely contradiction resolution rather than claim extraction) and empirical validation of the $O(n \cdot k)$ complexity bound.

Experiment 3: Finality Robustness. Inject adversarial evidence patterns—spike-and-drop confidence, oscillating claims, stale evidence exceeding temporal validity, trivial empty scopes—and measure false finality rate (RESOLVED despite unresolved contradictions), gate trigger frequency, and ESCALATED rate. Expected outcome: Gate C (oscillation detection) catches patterns that simple thresholds miss; Gate B (evidence freshness) blocks finality on stale data.

Experiment 4: Multi-Level Governance. Define three governance levels: operational (permissive mode), compliance (supervised mode), regulatory (deterministic mode). Run the M&A scenario with cross-level escalation rules. Measure decision distribution across levels, escalation frequency, and time-to-finality per level. Expected outcome: >80% of decisions resolved at the operational level, with rare but critical regulatory-level decisions that block finality until human resolution.

Experiment 5: Coverage-Autonomy Trade-off. Run identical document sets through all three governance modes. Measure claims accepted, contradictions resolved autonomously, human escalations, and convergence rate α. Map results to SECP's coverage-autonomy framework: permissive mode $\sim$ scalar aggregation (high coverage, lower confidence), deterministic mode $\sim$ hard veto (low coverage, high confidence), supervised mode $\sim$ intermediate. Expected outcome: the Lyapunov convergence rate α differs characteristically across modes, providing a formal metric for the trade-off that SECP identified empirically.

11 Limitations and Future Work

Scalability: Validation at 50 claims with 30% logical overlap. Production scale (thousands of claims, hundreds of agents) requires semantic graph optimization (approximate nearest-neighbor search, event log sharding, compiled governance rules).

Heterogeneous agents: Current agents are LLM-based. Extending to symbolic reasoners, neural networks, and human experts requires standardized agent interfaces and cross-modality convergence criteria.

Real LLM trajectories: Convergence benchmarks use synthetic data. Validation with real LLM reasoning traces (with their inherent stochasticity) remains future work. Theorem 1's precondition—that approved proposals always reduce some gap—may not hold under LLM stochasticity without additional safeguards.

Byzantine agents: Current convergence proofs assume agents make progress. Handling adversarial or faulty agents requires Byzantine-tolerant consensus machinery [3, 2, 9]. Integrating SECP's BFT protocols with our Lyapunov convergence tracking is a natural next step.

Formal proofs: Theorem 1 is a proof sketch. Mechanized verification in a proof assistant (Coq, Lean) would strengthen the convergence guarantee.

Protocol self-modification: Our governance rules are static. SECP demonstrates that bounded self-modification is feasible under formal invariants [3]. Extending our declarative governance to support constrained self-evolution—where rules adapt while preserving the monotonicity invariants of Definition 1—is an open research direction.

Multi-scope coordination: Current validation is single-scope. Enterprise deployments will require cross-scope governance and convergence.

12 Conclusion

We have presented a paradigm for autonomous agent coordination based on declarative governance over shared immutable context.

The theoretical contribution is twofold. First, Whitney-stratified constraint manifolds formalize how governance rules progressively reduce a high-dimensional enterprise decision space to discrete terminal states. Second, Fisher–Rao information geometry describes the dynamics of agent convergence toward those states. The Lyapunov disagreement function $V(t)$, with its five-gate finality predicate and conditional monotonicity theorem, provides formal convergence guarantees that are absent from existing agent coordination frameworks.

The practical contribution is an architecture where agents reason independently over shared context, propose transitions evaluated by declarative rules, and converge through formal mechanisms—or route to structured human review when autonomous consensus is insufficient. Unlike existing frameworks that wire agents into predefined topologies, coordination patterns emerge from policy.

Our work is complementary to recent advances in Byzantine-tolerant agent coordination. Where de la Chica Rodriguez and Vera Díaz [3] demonstrate that bounded protocol self-modification is feasi-

ble under formal invariants, we demonstrate that Lyapunov-based convergence tracking provides the formal foundation for multi-iteration deployment of such mechanisms. Their work addresses Byzantine tolerance; ours addresses convergence guarantees, audit infrastructure, and formal finality—together covering the requirements for deploying autonomous agents in regulated environments. The five proposed experimental protocols (Section 10) are designed to bridge these contributions empirically.

As LLM agents become increasingly capable of reasoning, planning, and hypothesis generation, the need for governance-based coordination—rather than pipeline-based orchestration—will only grow.

Acknowledgments

This work was supported by Deal ex Machina SAS. We thank the communities behind PostgreSQL, NATS, OpenFGA, DSPy, and Mastra for foundational infrastructure.

References

[1] Shun-ichi Amari and Hiroshi Nagaoka. *Methods of Information Geometry*, volume 191 of *Translations of Mathematical Monographs*. American Mathematical Society, 2000.

[2] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–186, 1999.

[3] Jose Manuel de la Chica Rodriguez and Juan Manuel Vera Díaz. Self-evolving coordination protocol in multi-agent AI systems: An exploratory systems feasibility study. `https://arxiv.org/abs/2602.02170`, 2025. arXiv:2602.02170. AI Lab, Grupo Santander.

[4] Marco Dorigo, Mauro Birattari, and Thomas Stutzle. Ant colony optimization: Overview and recent advances. *Handbook of Metaheuristics*, pages 311–351, 2019.

[5] David Fisher. Superrigidity, arithmeticity, normal subgroups: Results, ramifications and directions. `https://arxiv.org/abs/2003.02956`, 2020. arXiv:2003.02956.

[6] Suyash Gaurav, Jukka Heikkonen, and Jatin Chaudhary. Governance-as-a-service: A multi-agent framework for AI system compliance and policy enforcement. `https://arxiv.org/abs/2508.18765`, 2025. arXiv:2508.18765.

[7] Brendan Gho, Suman Muppavarapu, Afnan Shaik, Tyson Tsay, Atharva Mohan, James Begin, Kevin Zhu, Archana Vaidheeswaran, and Vasu Sharma. From competition to coordination: Market making as a scalable framework for safe and aligned multi-agent LLM systems. `https://arxiv.org/abs/2511.17621`, 2025. arXiv:2511.17621.

[8] Shadaj Laddad, Conor Power, Alvin Cheung, and Joseph M. Hellerstein. Katara: Synthesizing CRDTs with verified lifting. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2022.

[9] Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.

[10] LangChain Team. LangGraph: Building stateful, multi-actor applications with LLMs. `https://github.com/langchain-ai/langgraph`, 2024.

[11] Grigory A. Margulis. Discrete subgroups of semisimple Lie groups. *Ergebnisse der Mathematik und ihrer Grenzgebiete*, 1975. Superrigidity theorem announced.

[12] Grigory A. Margulis. Arithmeticity of the irreducible lattices in the semi-simple groups of rank greater than 1. *Inventiones Mathematicae*, 76:93–120, 1984.

[13] Grigory A. Margulis. *Discrete Subgroups of Semisimple Lie Groups*. Springer-Verlag, 1991.

[14] João Moura. CrewAI: Framework for orchestrating role-playing autonomous AI agents. `https://github.com/joaomdmoura/crewAI`, 2024.

[15] Reza Olfati-Saber and Richard M. Murray. Consensus problems in networks of agents with switching topology and time-delays. *IEEE Transactions on Automatic Control*, 49(9):1520–1533, 2004.

[16] Ruoming Pang et al. Zanzibar: Google's consistent, global authorization system. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC)*, pages 33–46, 2019.

[17] Markus J. Pflaum. *Analytic and Geometric Study of Stratified Spaces*, volume 1768 of *Lecture Notes in Mathematics*. Springer-Verlag, 2001.

[18] Chaoyi Ruan, Yiliang Wang, et al. Reaching agreement among reasoning LLM agents. `https://arxiv.org/abs/2512.20184`, 2025. arXiv:2512.20184.

[19] Yuntian Shen et al. CodeCRDT: Observation-driven coordination for multi-agent LLM code generation. `https://arxiv.org/abs/2510.18893`, 2025. arXiv:2510.18893.

[20] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 2000.

[21] Alfred Tarski. A decision method for elementary algebra and geometry. *RAND Corporation*, 1951. 2nd edition.

[22] Hassler Whitney. Tangents to an analytic variety. *Annals of Mathematics*, 81(3):496–549, 1965.

[23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

A Implementation Details

This appendix provides implementation-level specifications for the system described in Section 6. All examples are drawn from the reference implementation.[1]

A.1 Event Log Schema

Every event in the append-only Context WAL follows this structure:

```
{
  "timestamp": "2025-08-15T10:32:00Z",
  "agent_id": "facts_extraction_v2",
  "event_type": "claim_emitted",
  "payload": {
    "entity": "NovaTech AG",
    "relation": "annual_recurring_revenue",
    "value": "EUR 50M",
    "confidence": 0.92,
    "source_doc": "analyst_briefing.pdf"
  }
}
```

The semantic graph is stored in Postgres + pgvector, with monotonic upserts enforced at the database level.

A.2 Bitemporal Query Model

Every node and edge carries two independent time axes:

- **Valid time** (`valid_from`, `valid_to`): when a fact holds in the real world.
- **Transaction time** (`recorded_at`, `superseded_at`): when the system learned or corrected the fact.

The current view filter: $\texttt{superseded_at IS NULL} \wedge (\texttt{valid_to IS NULL} \vee \texttt{valid_to} > \texttt{now()})$.

An *evidence schema* declares required evidence types and maximum staleness per domain. The finality evaluator uses this schema to block finality when required evidence is missing or has exceeded its temporal validity.

[1] `https://github.com/DealExMachina/swarm-of-governed-agents`

A.3 Governance Configuration

```
governance:
  approval_modes:
    YOLO:    # Automatic approval for valid transitions
    MITL:    # All proposals route to human review
    MASTER:  # Deterministic rule-based, no LLM rationale

  drift_rules:
    - type: contradiction
      severity: [medium, high]
      action: open_investigation
    - type: goal_misalignment
      severity: [medium, high]
      action: request_goal_refresh
    - type: entropy
      severity: high
      action: halt_and_review

  transitions:
    - from: DriftChecked
      to: ContextIngested
      blocking_condition: critical_drift
      override: requires_human_authorization
```

The three-node state cycle operates as: `ContextIngested` → `FactsExtracted` → `DriftChecked` → `ContextIngested`, with epoch-based compare-and-swap ensuring atomicity.

A.4 Convergence Parameters

Dimension	Target	Weight	Meaning
Claim confidence	≥ 0.85	0.30	Active claims at sufficient confidence
Contradiction resolution	$= 0$	0.30	Zero unresolved contradictions
Goal completion	≥ 0.90	0.25	Completion ratio of declared goals
Risk score inverse	< 0.20	0.15	Residual risk below threshold

Table 4: Convergence dimensions with weights and targets.

Finality state triggers:

State	Trigger	Action
RESOLVED	Score ≥ 0.92 + monotonic gate ($\beta = 3$)	Automatic completion
ACTIVE	Score < 0.92 or gate not satisfied	Continue processing
ESCALATED	Risk ≥ 0.75 or ≥ 3 contradictions or $\alpha < -0.05$	Human intervention
BLOCKED	≥ 5 idle cycles + ≥ 300s without updates	Stalled
EXPIRED	No activity for 30 days	Process expired
HITL Review	Score $\in [0.40, 0.92)$ + plateau ($\tau = 3$)	Structured human decision

Table 5: Finality state triggers with implementation parameters.

Plateau detection uses EMA of progress ratio:

$$\text{EMA}(t) = 0.3 \cdot \text{progress_ratio}(t) + 0.7 \cdot \text{EMA}(t-1)$$

Plateau declared when EMA < 0.01 for $\tau = 3$ consecutive rounds.

Convergence threshold: $\epsilon = 0.005$.

A.5 Dynamic Scaling

The hatchery subsystem uses M/M/c queueing theory with KEDA-style lag monitoring to determine optimal agent instance counts per role. Erlang-style supervisors (one-for-one restart) handle agent failures. This enables elastic scaling without architectural changes.

A.6 Testing and Benchmarks

197 unit tests across 26 files covering: state graph CAS transitions (11), convergence tracker stability (28), finality evaluator logic (11), governance agent modes (18), embedding pipeline (11), and supporting infrastructure.

7 convergence benchmarks validating mathematical properties independently of infrastructure:

Scenario	Rounds	Outcome
Steady convergence ($\sim$5%/round)	15	Monotonic, converging, ETA ≈ 0
Plateau at 0.70 ($\pm$0.002 oscillation)	10	Stagnation detected
Spike-and-drop (0.95 $\rightarrow$ 0.70)	–	Monotonicity gate blocks premature finality
Divergence (contradictions increase)	–	Negative α, zero forward progress
One-dimension bottleneck	–	Pressure identifies blocker dimension
Fast convergence ($\geq$0.92 in 3 rounds)	3	No false plateau during rapid improvement
Empty graph (no claims/goals)	–	Safe defaults, no division by zero

Table 6: Convergence benchmarks validating formal properties.